

PALM: Synergizing Program Analysis and LLMs to Enhance Rust Unit Test Coverage

Bei Chu¹, Yang Feng¹, Kui Liu², Hange Shi¹,
Zifan Nan², Zhaoqiang Guo² and Baowen Xu¹

¹State Key Laboratory for Novel Software Technology, Nanjing University

²Software Engineering Application Technology Lab, Huawei



The Challenge

Traditional Methods (SBST, Concolic)

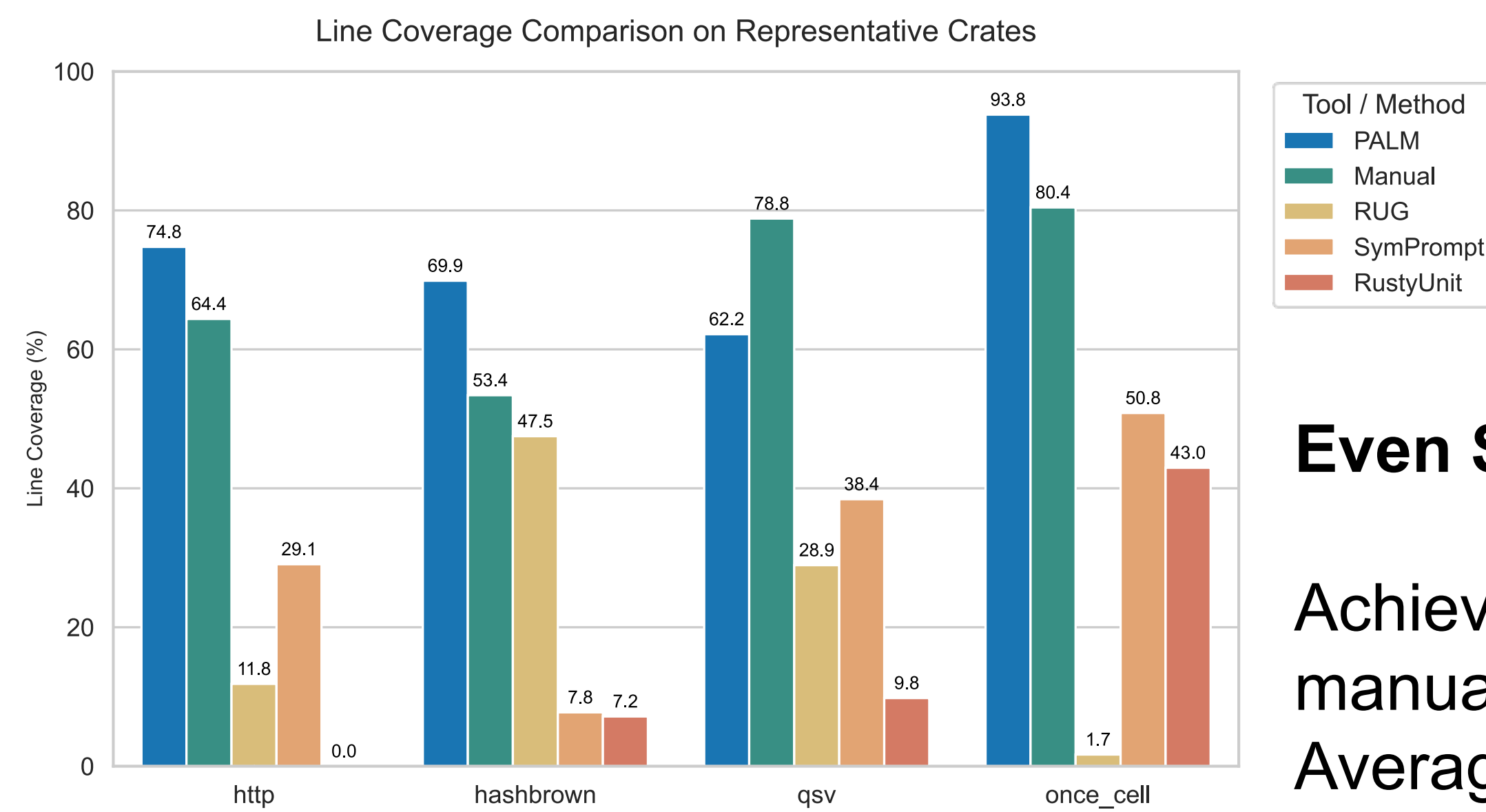
- Struggle with Rust's unique features (ownership, concurrency).
- Suffer from **Path Explosion** and complex environment modeling.

Existing LLM Methods

- Rely on generic, **fixed prompts** with no specific guidance.
- Result in low compilation rates and poor branch coverage.

Our Goal: Generate **high-coverage, compilable** Rust tests by **intelligently guiding** LLMs.

Key Results



PALM's average line coverage (72.3%) is comparable to human effort (70.9%).

Even Stronger on Branch Coverage

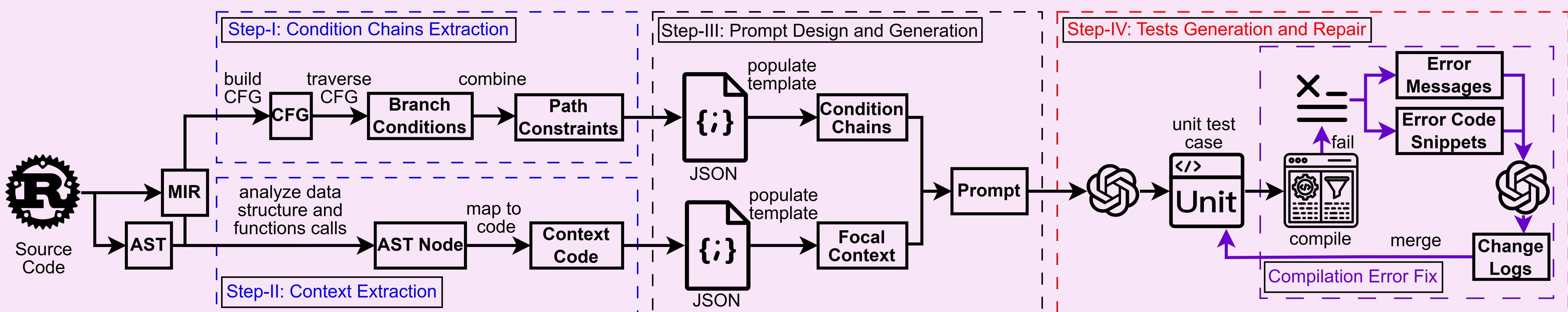
Achieved higher branch coverage than manual tests in **10 out of 15** projects.

Average Branch Coverage:

PALM: **69.5%**

Manual: **66.5%**

The PALM Framework



1. Path & Constraint Extraction

Analyze Rust's Mid-level IR (MIR) to extract precise execution paths and their constraints.

Output: A "Condition Chain" for each path.

2. Path-Guided Prompting

Construct a specific prompt for each path, embedding the Condition Chain to guide the LLM.

Key Input: "Generate a test satisfying ..."

3. Guided Test Generation

The LLM focuses on the single, simple task of generating a test case that satisfies the given path constraints.

Output: A targeted, path-specific unit test.

4. Iterative Repair

If a test fails to compile, feed the compiler error back to the LLM to automatically generate a fix.

Loop: Compile → Get Error → LLM Fix → Re-compile

Proven in the Wild

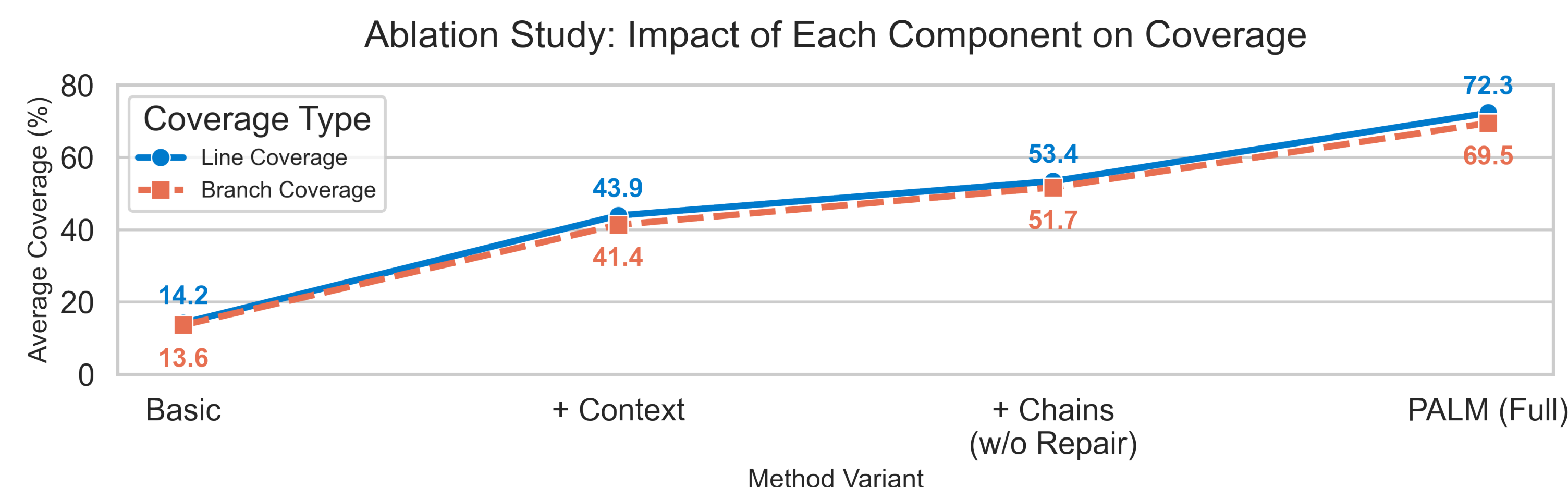
94.1%
Acceptance Rate

Submitted **91** PALM-generated tests to **8** popular open-source projects.
80 tests were Accepted & Merged by maintainers.

80 accepted / 85 resolved; 6 pending excluded.

Contributions to: hashbrown, once_cell, bytes, rand, etc.

Ablation Study

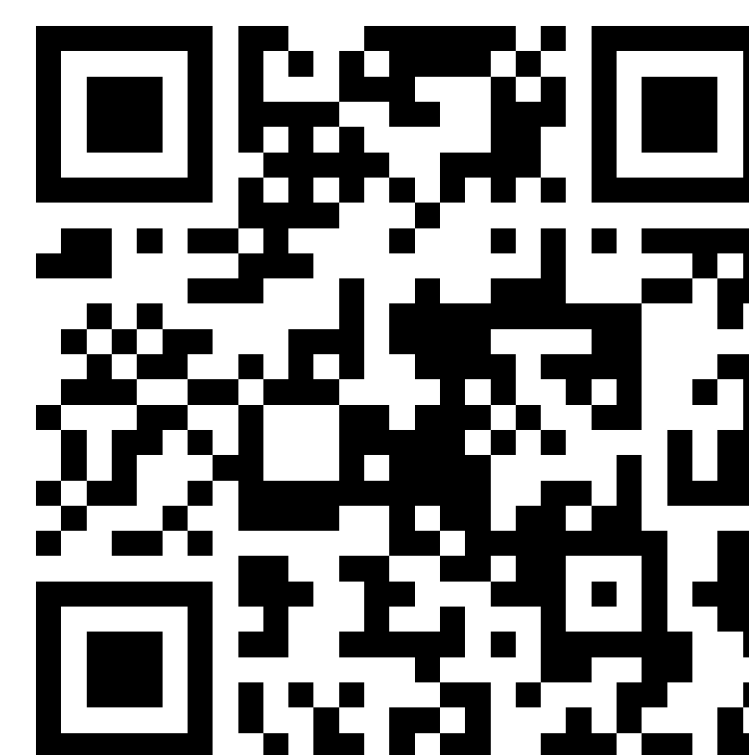


Key Finding: Both **Path-Specific Guidance (Condition Chains)** and **Automated Repair** are crucial for achieving high performance.

Conclusion & Contact

Conclusion

- **PALM:** A novel approach synergizing program analysis and LLMs.
- **Path-Guided Prompting & Iterative Repair** are the keys to its success, significantly improving coverage and compilation rates.
- **Proven Practicality:** Achieved SOTA results and was validated by a high pull request acceptance rate in major open-source projects.



Scan for paper



PALM materials

Find Out More

Bei Chu
beichu@smail.nju.edu.cn